

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-aa8e3743f26e18b54.jsonl`  
- SHA-256: `87a7c5919a112a59f17da972aff024a475fd38f5b60f7f6f74ca4e41cd1b6ce7`  
- Source modified: `2026-07-06T19:19:05+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T19:18:19.570Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).  
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.  
No code changes. Read the ADR:  
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:  
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').  
Relevant current code (on main): src/tg\_video\_filter/models.py (FilterConfig, ContentItem, Source, Policy),  
src/tg\_video\_filter/db.py (load\_filter\_config/save\_filter\_config shims, sources.watched gate should\_ingest\_source, delivery\_attempts/record\_delivery), src/tg\_video\_filter/telethon\_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery\_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content\_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.  
Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,  
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).  
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).  
FINDING (design): PolicyContext has no client/message handle ? Tier 3 and Tier 4 cannot perform the I/O the ADR assigns them  
severity high | section Decision ? Stage interface (PolicyContext); Tier 3; Tier 4  
summary: PolicyContext carries only (content\_item, source, policy). Tier 3 does 'light I/O' and Tier 4 does ffprobe/download, but neither the ContentItem nor Source gives a handle to fetch bytes or re-query Telegram, so the I/O tiers have no way to do I/O within the defined contract.  
why\_it\_matters: Tier 4 is defined as 'ffprobe, perceptual hash, downloaded content scan' with a byte budget, and the evaluate\_pipeline signature only passes PolicyContext. To download you need the live Telethon client + the Message (or at least media\_ref resolvable to a download). ContentItem.media\_ref is 'provider media reference' (a string, NULL in Phase 1 for the live path) ? not a downloadable handle, and there is no client on the context. F5 cannot be implemented against the contract as written. Even Tier 3 'caption text scanning' needs text that isn't in the context in usable form (see next finding).  
evidence: ADR PolicyContext dataclass = content\_item, source, policy only.  
evaluate\_pipeline(ctx, stages) passes only ctx. models.py ContentItem.media\_ref is a str|None with no client; the live handler (telethon\_client.py) holds the TelegramClient + Message but PolicyContext exposes neither.  
rec: Add a fetch/client capability to the contract for I/O tiers ? e.g. an optional `fetcher`/`client` handle or a `download(budget)` callable on PolicyContext, or split the protocol so I/O stages receive an extra I/O-capable context. Otherwise explicitly scope Tier 3/4 to metadata-only and move download/ffprobe out of this ADR.

## 2. assistant (2026-07-06T19:18:22.160Z)

I'll verify this finding by reading the ADR and the relevant code.

## 3. user (2026-07-06T19:18:23.758Z)

# ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **\*\*Status\*\***: Proposed
- **\*\*Date\*\***: 2026-07-06
- **\*\*Decision owner\*\***: project owner
- **\*\*Tracking issue\*\***: #31 (Track F of the provider-neutral umbrella, #25)
- **\*\*Depends on\*\***: ADR 0011 (Track B ? domain schema, `policies` table)

## Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **\*\*flat\*\***: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **\*\*policy pipeline\*\*** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

## What we have today (post-Track-E)

```
| Concept | Current state |
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`.
```

| `policies` table | One row per account (is\_default=1). `config\_json` is the `FilterConfig` blob. |  
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffmpeg were deferred to v2 in the original plan. |

## What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

## Decision

### Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                Policy Pipeline                                ?
?                                                                ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)              ?
?   duration, resolution, size, codec, mime, GIF/round          ?
?   ? extends today's FilterEngine                             ?
?                                                                ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)            ?
?   source allowlist, keyword match, channel topic tags         ?
?   ? user-expressed interests filter the stream                ?
?                                                                ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)       ?
?   URL/domain reputation, file metadata sanity,                ?
?   caption text scanning                                       ?
?   ? cheap checks; no download                                 ?
?                                                                ?
? Tier 4 ? Deep inspection  (opt-in, heavy I/O)                 ?
?   ffmpeg, perceptual hash, downloaded content scan           ?
?   ? gated behind explicit user opt-in + budget               ?
????????????????????????????????????????????????????????????
...
```

### Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem      # from ADR 0011
    source: Source                  # the source it came from
    policy: Policy                  # the owning policy config

@dataclass
class StageResult:
    passed: bool
    reason: str                    # human-readable explanation
    stage_name: str                # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict) # for explainability UI
```

```
class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
    ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult`` list is the **explanation trace** ? every decision is auditable.

## Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json`` under a `topics`` key:

```
```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any" // "any" = match any include topic; "all" = match all
  }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

## Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist`` (an explicit list of trusted `provider_source_ids`).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint`` across accounts (already handled by `content_items.dedup_key``; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata`` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}``).

## Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true`` in `config_json``) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

## Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
```

```

    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon\_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

## Schema ? minimal changes

**\*\*No schema migration\*\*** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

## Implementation phasing (future issues)

| Phase         | Scope                   | Dependencies                                                                         |
|---------------|-------------------------|--------------------------------------------------------------------------------------|
| ---           | ---                     | ---                                                                                  |
| <b>**F1**</b> | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler   None |
| <b>**F2**</b> | Topic preferences       | `topics` config parsing, source title/caption keyword matching   F1                  |
| <b>**F3**</b> | Source allowlist        | `allowlist` config, skip non-allowlisted sources   F1                                |
| <b>**F4**</b> | URL/domain reputation   | Domain blocklist, caption link extraction   F1                                       |
| <b>**F5**</b> | Deep inspection         | ffprobe/perceptual hash, download budget enforcement   F1, ADR 0003 waiver           |
| <b>**F6**</b> | Policy decisions table  | Audit trail storage, explainability UI   F1                                          |
| <b>**F7**</b> | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace   F6                  |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the **\*\*design contract\*\*** the phases code against.

## Consequences

### **\*\*Positive\*\***

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

### **\*\*Negative\*\***

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same

- interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

## Alternatives considered

- **\*\*Do nothing; keep flat FilterConfig.\*\*** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **\*\*Inline each stage in the handler\*\*** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **\*\*Use a rules engine / DSL.\*\*** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **\*\*Bundle all phases into a single PR.\*\*** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

## Open questions for the decision owner

1. **\*\*`StageResult.metadata` shape\*\*** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **\*\*Policy decisions table\*\*** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **\*\*Topic matching\*\*** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **\*\*Deep inspection budget\*\*** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

## References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

### 4. assistant (2026-07-06T19:18:27.329Z)

Now let me examine the actual code to verify the finding's claims about the code and other ADRs.

### 5. user (2026-07-06T19:18:29.146Z)

```
"""Pydantic data models for users, channels, videos, and filter config.
```

```
These models are the in-memory representation used by the repository layer
(`db.py`) and the filter engine (`filters/`). They are deliberately
decoupled from Telethon's own types so the filter engine stays pure and
unit-testable without a live Telegram connection.
"""
```

```

from __future__ import annotations

from datetime import UTC, datetime
from enum import Enum

from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator


def _utcnow() -> datetime:
    """Timezone-aware UTC 'now', used as a default factory."""
    return datetime.now(UTC)


class DeliveryMode(str, Enum):
    """How a matched video is delivered to the user.

    - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
    - ``LINK``: send a text message with a t.me link + metadata summary.
    - ``BOTH``: forward the video AND send the link message.
    """

    FORWARD = "forward"
    LINK = "link"
    BOTH = "both"


class Account(BaseModel):
    """A product identity ? the owner of config, channels, routes, and content.

    Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
    is the bridge used by the Bot API front-door (``from.id``) and by the
    legacy userbot path; it is nullable so future providers that don't have a
    Telegram id (e.g. WhatsApp) can still own an ``Account``.
    """

    model_config = ConfigDict(frozen=True)

    id: int = Field(description="Stable surrogate PK; the owner key for all downstream tables")
    telegram_id: int | None = Field(
        default=None,
        description="Telegram user id (bot from.id OR userbot account id). UNIQUE when set.",
    )
    provider: str = Field(default="telegram", description="Provider namespace; future: 'whatsapp'")
    display_name: str | None = None
    created_at: datetime = Field(default_factory=_utcnow)


class TelegramSession(BaseModel):
    """An optional MTPROTO (Telethon) session linked to an :class:`Account`.

    A bot-only user has an ``Account`` row but **no** ``TelegramSession`` row;
    that is what makes them structurally invisible to ``manager.run``
    (ADR 0010). ``session_path`` is data-driven so the file-location
    convention is explicit rather than derived from the PK at runtime.
    """

    model_config = ConfigDict(frozen=True)

    account_id: int = Field(description="FK ? accounts.id; PK (one session per account in v1)")
    phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
    session_path: str = Field(description="File-backed Telethon session path (user_<id>.session)")
    enabled: bool = Field(
        default=True,

```

```

        description="If False, the manager skips this session (not the account)",
    )
    created_at: datetime = Field(default_factory=_utcnow)
    last_login_at: datetime | None = None

class User(BaseModel):
    """Deprecated: legacy v1 model conflating identity + MTPROTO session.

    Retained as a backwards-compatible view over the new ``accounts`` +
    ``telegram_sessions`` tables (ADR 0010). New code should use
    :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
    maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
    and tests keep working without a flag day.
    """

    model_config = ConfigDict(frozen=True)

    id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy rows)")
    phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
    enabled: bool = Field(default=True, description="If False, the manager skips this session")
    created_at: datetime = Field(default_factory=_utcnow)

class Channel(BaseModel):
    """A channel/group a user is a member of. Populated lazily as messages arrive.

    Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
    code should use :class:`Source` directly; the ``db`` layer maps
    ``Channel`` ? ``Source`` so existing callers and tests keep working.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    user_id: int
    channel_id: int = Field(description="Telegram channel/chat id")
    title: str | None = None
    seen_first_at: datetime = Field(default_factory=_utcnow)

```

### --- provider-neutral domain models (ADR 0011, Track B) -----

#

**These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig`` shapes so the schema no longer leaks Telegram-specific ids into every future provider's code path. ``provider`` columns + provider-prefixed ``dedup\_key`` make cross-provider collisions structurally impossible. See ADR 0011 for the full rationale and the two decisions (Option 1A: tiered ``dedup\_key``; Option 2A: ``policies`` generalizes ``filter\_config``).**

```

class Source(BaseModel):
    """A configured place we ingest content from (ADR 0011).

    Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
    providers add their own source kinds). ``provider_source_id`` is the
    provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
    """

    model_config = ConfigDict(frozen=True)

```



```

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id (owner of the source)")
provider: str = Field(default="telegram", description="Provider namespace")
provider_source_id: str = Field(
    description="Provider-native source id (Telegram channel_id, ?)"
)
title: str | None = None
kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
watched: bool = Field(
    default=False,
    description="ADR 0009 selective scanning: ingest from this source iff True",
)
seen_first_at: datetime = Field(default_factory=_utcnow)
last_seen_at: datetime | None = None

class ContentItem(BaseModel):
    """A normalized content record discovered from any :class:`Source` (ADR 0011).

    Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
    (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
    NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
    ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
    wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    source_id: int = Field(description="FK ? sources.id")
    provider: str = Field(default="telegram", description="Provider namespace")
    provider_message_id: str = Field(
        description="Provider-native message id (Telegram message_id, ?)"
    )
    content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
    text: str | None = Field(default=None, description="Caption / post text (nullable in Phase
1)")
    link_url: str | None = Field(default=None, description="For link/article posts (Phase 1:
NULL)")
    media_ref: str | None = Field(default=None, description="Provider media reference")
    document_id: str | None = Field(
        default=None,
        description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
    )
    duration_s: int | None = None
    width: int | None = None
    height: int | None = None
    size_bytes: int | None = None
    mime_type: str | None = None
    dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per
account")
    matched: bool = False
    seen_at: datetime = Field(default_factory=_utcnow)

class Policy(BaseModel):
    """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).

    Generalizes the legacy per-account ``filter_config`` row so one account can
    own multiple policies (enables ADR 0009's per-route filter overrides) and so
    future enrichment/security stages grow into the same ``config_json`` column.
    Exactly one policy per account carries ``is_default=True``; the legacy
    ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
    """

```

```

model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id")
name: str = Field(description="Policy name; 'default' for the account default policy")
config_json: str = Field(
    description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
)
is_default: bool = Field(
    default=False, description="Exactly one per account (UNIQUE partial index)"
)
created_at: datetime = Field(default_factory=_utcnow)
updated_at: datetime = Field(default_factory=_utcnow)

```

### --- Track C: destinations, routing, delivery state (ADR 0011) -----

#  
**These tables were created empty by Track B (\_SCHEMA); Track C populates destinations (backfilled from filter\_config\_deprecated.config\_json), writes delivery\_attempts (replacing the legacy videos.forwarded boolean), and defines the models for per-destination delivery state. The source/destination invariant (a channel is either source or destination, never both) is enforced at the app level by future selective-scanning logic (Track E, #30).**

```

class Destination(BaseModel):
    """A delivery target ? where matched content is sent (ADR 0011, Track C).

    Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
    can have multiple named output channels. Backfilled from the existing
    per-account ``delivery_target``; the default destination is the target used
    when no explicit routing rule matches.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    provider: str = Field(default="telegram", description="Provider namespace")
    target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
    kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
    title: str | None = None
    is_output: bool = Field(
        default=False,
        description="ADR 0009: distinguishes output channels from ingest sources",
    )
    created_at: datetime = Field(default_factory=_utcnow)


class RoutingRule(BaseModel):
    """A rule that maps a source through a policy to a destination (ADR 0009).

    Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
    ``policy_id`` is NULL, the account's default policy is used. When multiple
    rules match, the highest ``priority`` wins.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")

```

```

account_id: int = Field(description="FK ? accounts.id")
source_id: int = Field(description="FK ? sources.id")
policy_id: int | None = Field(
    default=None,
    description="FK ? policies.id; NULL means use account default policy",
)
destination_id: int = Field(description="FK ? destinations.id")
enabled: bool = Field(default=True)
priority: int = Field(default=0, description="Higher wins when multiple rules match")
created_at: datetime = Field(default_factory=_utcnow)

class DeliveryStatus(str, Enum):
    """Per-destination delivery state, replacing the legacy binary ``forwarded``."""

    PENDING = "pending"
    DELIVERED = "delivered"
    FAILED = "failed"
    RETRYING = "retrying"

class DeliveryAttempt(BaseModel):
    """Per-destination delivery record ? idempotent via UNIQUE index.

    Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
    ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
    incremented and ``status``/``last_error`` are updated.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    content_item_id: int = Field(description="FK ? content_items.id")
    destination_id: int = Field(description="FK ? destinations.id")
    route_id: int | None = Field(default=None, description="FK ? routing_rules.id (nullable)")
    status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
    attempt_count: int = Field(default=0, description="Incremented on each retry")
    last_error: str | None = None
    created_at: datetime = Field(default_factory=_utcnow)
    delivered_at: datetime | None = None
    next_retry_at: datetime | None = None

class FilterConfig(BaseModel):
    """Per-user filter configuration.

    v1 rules: duration bounds, minimum height (resolution), and max file size.
    Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
    new field later is backwards-compatible.
    """

    # Duration bounds in seconds. None means unbounded on that side.
    min_duration_s: int | None = Field(default=5, ge=0)
    max_duration_s: int | None = Field(default=7200, ge=0)

    # Resolution: a video passes if its height >= target_height.
    # None disables the resolution rule.
    target_height: int | None = Field(default=1080, ge=1)

    # Max file size in mebibytes. None disables the size rule.
    max_size_mb: int | None = Field(default=2048, ge=1)

    # Content-type exclusions. GIFs and round videos are excluded by default
    # (their duration/dimensions are usually incidental to real video filters);
    # silent videos are off by default since some legit clips have no audio.

```

```

# Existing users' stored configs are unaffected ? the DB JSON blob overrides
# these defaults, so this only changes the behaviour for fresh installs.
exclude_gif: bool = Field(
    default=True,
    description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
)
exclude_round: bool = Field(
    default=True,
    description="If True, exclude circular video messages (round_message=True)",
)
exclude_nosound: bool = Field(
    default=False,
    description="If True, exclude videos without an audio track (nosound=True)",
)

# How matched videos are delivered: forward the video, send a link, or both.
# Stored in the same JSON blob, so adding this field is backwards-compatible
# (existing rows without it default to FORWARD).
delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)

# Where matched videos / link messages are sent. Accepts:
# - "me" (default): Saved Messages ? exact v1.1 behaviour.
# - "@username": a public channel the account is a member of.
# - "-100..." or a bare int-as-string: a numeric Telegram chat id
#   (private channels without a username).
# The string is passed verbatim to Telethon's forward_messages / send_message.
# See ADR 0005 for rationale and access requirements.
delivery_target: str = Field(
    default="me",
    min_length=1,
    description="Telethon entity to deliver to: 'me', '@username', or a chat id",
)

@field_validator("delivery_target", mode="before")
@classmethod
def _coerce_delivery_target(cls, v: object) -> str:
    """Accept int chat ids (e.g. -1001234567890) and coerce to str.

    Telethon accepts both str and int as an entity, but storing as str keeps
    the JSON blob uniform and lets '@username' / 'me' / numeric ids share
    one field type.
    """
    if v is None:
        return "me"
    if isinstance(v, int):
        return str(v)
    return str(v)

@model_validator(mode="after")
def _check_duration_bounds(self) -> FilterConfig:
    """Ensure min_duration_s <= max_duration_s when both are set."""
    if (
        self.min_duration_s is not None
        and self.max_duration_s is not None
        and self.min_duration_s > self.max_duration_s
    ):
        raise ValueError(
            f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
            f"({self.max_duration_s})"
        )
    return self

class VideoRecord(BaseModel):
    """A seen video message + its metadata. Drives dedup and future /search.

```

Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).  
New code should use :class:`ContentItem`; the ``db`` layer maps  
``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.  
"""

```
model_config = ConfigDict(frozen=True)

id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
user_id: int
channel_id: int = Field(description="Origin Telegram channel id")
message_id: int = Field(description="Telegram message id within the channel")
document_id: int | None = Field(
    default=None,
    description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
)

# Metadata extracted from the Telethon message (may be None if absent).
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None

matched: bool = False
forwarded: bool = False
seen_at: datetime = Field(default_factory=utcnow)

@property
def dedup_key(self) -> tuple[int, int, int]:
    """The natural key used for deduplication in the ``videos`` table."""
    return (self.user_id, self.channel_id, self.message_id)

@property
def content_fingerprint(self) -> tuple[int, int, int, int] | None:
    """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).

    Returns None if any field is missing (cannot fingerprint).
    """
    if None in (self.size_bytes, self.duration_s, self.height, self.width):
        return None
    return (self.size_bytes, self.duration_s, self.height, self.width) # type: ignore[return-value]
```

## Default filter config shared by the bootstrap onboarding step.

```
DEFAULT_FILTER_CONFIG = FilterConfig()
```

### 6. user (2026-07-06T19:18:30.156Z)

(Bash completed with no output)

### 7. assistant (2026-07-06T19:18:34.423Z)

The finding's claims about `ContentItem` and `PolicyContext` are confirmed by reading the code. Let me verify a couple of nuances: (1) whether `media\_ref` could serve as a downloadable handle, (2) whether the telethon\_client actually holds the client/Message but doesn't pass it, and (3) whether the ADR anywhere addresses passing a client.

### 8. user (2026-07-06T19:18:36.672Z)

"""Telethon ingestion: client factory, event handler, and metadata extraction.

Architecture (see docs/design.md §5):

```
NewMessage event
-> extract_video_meta(message) # pure, unit-tested
-> dedup via db.get_video()
-> FilterEngine.evaluate(meta)
-> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
```

The metadata extraction is deliberately a **pure** function taking a Telethon `Message` and returning a `VideoMeta | None`. This isolates the only piece of logic that's worth unit-testing (everything else is Telethon/async glue).

```
from __future__ import annotations

import logging
import sqlite3
from pathlib import Path
from typing import Protocol

import aiosqlite
from telethon import TelegramClient
from telethon.events import NewMessage
from telethon.tl.types import (
    Document,
    DocumentAttributeAnimated,
    DocumentAttributeVideo,
    Message,
    MessageMediaDocument,
)

from tg_video_filter import db
from tg_video_filter.commands import ENGINE_KEY
from tg_video_filter.delivery import build_link_text, deliver
from tg_video_filter.filters import FilterEngine, VideoMeta
from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord

log = logging.getLogger(__name__)
```

## --- pure metadata extraction -----

```
def extract_video_meta(message: Message) -> VideoMeta | None:
    """Extract VideoMeta from a Telethon Message, or None if not a video.
```

Returns `None` when the message has no video/document media. Returns a `VideoMeta` (with fields possibly `None`) for a video whose individual attributes are missing? the filter engine decides how to treat gaps.

Telegram stores video metadata in two places:

- `Document`: `size` (bytes), `mime_type` (str), `attributes` (list)
- `DocumentAttributeVideo` (inside `attributes`): `duration` (float s), `w` (width), `h` (height)

Note: we inspect `message.media` directly rather than the `.video` convenience property, because the latter only exists on Telethon's *custom* Message wrapper (applied at runtime) and is `None` on the raw TL Message used in tests. A Document is considered a video iff it contains a `DocumentAttributeVideo`.

```
"""
media = getattr(message, "media", None)
```

```
# Real Telegram messages wrap documents in MessageMediaDocument.
# Unwrap it to get the actual Document.
if isinstance(media, MessageMediaDocument):
```

```

media = media.document

if not isinstance(media, Document):
    return None

# Only treat this as a video if the document carries a video attribute.
has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in media.attributes)
if not has_video_attr:
    return None

meta = VideoMeta(
    size_bytes=media.size,
    mime_type=media.mime_type or None,
)

# Find the video attribute for duration/dimensions and round/nosound flags.
video_update: dict = {}
for attr in media.attributes:
    if isinstance(attr, DocumentAttributeVideo):
        if attr.duration:
            video_update["duration_s"] = int(round(attr.duration))
        if attr.w:
            video_update["width"] = attr.w
        if attr.h:
            video_update["height"] = attr.h
        video_update["is_round"] = attr.round_message # may be None
        video_update["nosound"] = attr.nosound # may be None
        break

# Check for GIF (DocumentAttributeAnimated present).
is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)

meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
return meta

```

## --- client factory -----

```

def build_client(
    session: TelegramSession, settings: SettingsLike
) -> TelegramClient: # pragma: no cover
    """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).

    The session path is read from the ``telegram_sessions.session_path`` column
    (data-driven, no longer derived from the PK at runtime) and resolved
    against ``settings.sessions_dir``. The directory is created if missing.
    """
    sessions_dir = Path(settings.sessions_dir)
    sessions_dir.mkdir(parents=True, exist_ok=True)
    # session_path is stored as a basename (e.g. "user_111.session"); join with
    # the runtime sessions_dir so the migration did not need to know the path.
    session_path = sessions_dir / session.session_path

    client = TelegramClient(
        str(session_path),
        api_id=settings.tg_api_id,
        api_hash=settings.tg_api_hash,
    )
    return client

class SettingsLike(Protocol):
    """Structural type for the subset of Settings the client factory needs.

```

Using a Protocol keeps the module importable in tests without dragging in the real Settings class. At runtime ``config.Settings`` satisfies this shape.

```
"""
```

```
tg_api_id: int
tg_api_hash: str
sessions_dir: str
```

## --- event handler wiring -----

```
def make_new_message_handler(
    *,
    user: User,
    conn: aiosqlite.Connection,
    engine_holder: dict[str, FilterEngine],
    client: TelegramClient,
    delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
    delivery_target: str = "me",
) -> _NewMessageCallback:
    """Return an async callback for ``events.NewMessage``.
```

The callback:

1. extracts VideoMeta + document\_id (skips if not a video),
2. dedups via two-tier content dedup (document.id, then metadata fingerprint), plus the existing (channel, message) instance dedup,
3. runs the filter engine (read from ``engine\_holder`` so config changes via ``/set`` take effect immediately),
4. inserts a VideoRecord row,
5. delivers a match to ``delivery\_target`` (see ADR 0005).

```
"""
```

```
async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
    message: Message = event.message # type: ignore[assignment]
    if message is None:
        return # Telethon edge case: event has no message (shouldn't happen)
    # Only process genuine channel/group posts and private messages we
    # receive ? outgoing messages (sent by our own account) are ignored.
    if getattr(message, "out", False):
        return

    meta = extract_video_meta(message)
    if meta is None:
        return # not a video

    channel_id = _peer_id(message.peer_id)
    message_id = message.id
    document_id = _extract_document_id(message)

    # --- watched gate (ADR 0009, Track E) -----
    # Opt-in: a source is ingested only when watched=True. An absent
    # sources row means the channel has never been seen ? treat it as
    # unwatched and lazily create a watched=0 row for /channels
    # discoverability (ADR 0009 lines 37-44, 130-131).
    if not await db.should_ingest_source(conn, user.id, channel_id):
        return # skip unwatched / unknown source

    # --- three-tier dedup (cheapest first) ---

    # Tier 0: same (user, channel, message) instance ? already processed.
    existing = await db.get_video(conn, user.id, channel_id, message_id)
    if existing is not None:
        return
```



```

# Tier 1: same Telegram document.id ? forwarded/cross-posted file.
if document_id is not None:
    existing = await db.get_video_by_document_id(conn, user.id, document_id)
    if existing is not None:
        log.info(
            "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
            "original_channel=%s original_msg=%s",
            user.id,
            document_id,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
    return

# Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
if (
    meta.size_bytes is not None
    and meta.duration_s is not None
    and meta.height is not None
    and meta.width is not None

```

## 9. user (2026-07-06T19:18:37.590Z)

```

61:?      ? cheap checks; no download      ?
64:?      ffprobe, perceptual hash, downloaded content scan      ?
71:Each stage is a pure function (or async function for I/O stages) that
90:      ""A single gate in the pipeline. Stateless; may be async for I/O stages.""
91:      async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
116:message caption keyword match, and (future) channel topic tags from
128:      configurable blacklist. Metadata-only ? no page fetch.
130:      `content_fingerprint` across accounts (already handled by
139:Deep inspection (ffprobe, perceptual hash, downloaded content scan) is
141:no-download principle. The opt-in is per-policy (`deep_inspect: true` in
143:concurrent downloads) enforced by the policy engine.
152:async def evaluate_pipeline(
165:The caller (telethon_client or future Bot API handler) records the
185:| **F1** ? Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire
into live-message handler | None |
189:| **F5** ? Deep inspection | ffprobe/perceptual hash, download budget enforcement | F1, ADR
0003 waiver |
200: framework ? no ad-hoc if-chains scattered across handlers.
227:- **Inline each stage in the handler** (no protocol). Rejected ? makes
254:- ADR 0003 ? metadata-first probing (the no-download principle that Tier

```

## 10. assistant (2026-07-06T19:18:55.122Z)

The verification is complete. Let me confirm my analysis of the key facts:

1. **PolicyContext** carries only `content_item`, `source`, `policy` ? confirmed (ADR Decision ? Stage interface).
2. **ContentItem.media\_ref** is `str | None`, description "Provider media reference" ? not a downloadable handle, no client. Confirmed in `models.py`.
3. **Source** carries no client either. Confirmed.
4. **evaluate\_pipeline(ctx, stages)** passes only `ctx`. Confirmed (ADR line 152-163).
5. The ADR nowhere mentions passing a `client`, `Message`, `fetcher`, or `download` callable into the contract. The grep hits for "download" are all in Tier 4 prose describing what it does, never in the interface. Confirmed.
6. The live handler (`telethon_client.py`) holds `client: TelegramClient` and `message: Message` in `on_new_message`, but `PolicyContext` exposes neither. Confirmed.

Now, adversarial checks to try to REFUTE:

- **Could `media_ref` be a downloadable handle?** In Telethon, to download you need

``client.download_media(message)`` or a ``Document`/file` reference resolvable against a live client (file references expire). A stored string ``media_ref`` alone is not downloadable without the client. And the ADR itself notes ``media_ref`` is NULL in Phase 1 for the live path (per `ContentItem` docstring, ``text`/`link_url`` land NULL; `media_ref` has no default population noted). So ``media_ref`` cannot serve as the I/O handle.

- **\*\*Is this out of scope because Tier 4 is "deferred"?\*\*** Tier 4 is deferred to F5, but the ADR explicitly states this ADR "only defines the design contract the phases code against" and that it "reserves the slot in the pipeline model so it doesn't require a schema migration." The contract as written (`PolicyContext + evaluate_pipeline` signature) is what F5 must code against ? and it has no I/O handle. So the gap is real for the contract even if implementation is deferred. Tier 3 (F4, caption link extraction ? "no download", metadata-only) actually does NOT need the client; the finding overstates Tier 3 slightly. But Tier 4 genuinely cannot be implemented against the contract.

- **\*\*Severity calibration:\*\*** The finding claims high because "F5 cannot be implemented against the contract as written." That is accurate ? Tier 4 as designed (`ffprobe/download` of bytes) is impossible with the given `PolicyContext`. But: it's a draft design doc, Tier 4 is explicitly deferred and reserved, and the fix is additive (add a `client/fetcher` to the context) ? no data loss, no broken F-phase that ships now. The contract gap is a genuine design gap that would block F5, which the finding's own rubric calls medium ("design gap that would break an F-phase"). Only "false claim / data-loss" is high. This is a real gap, not a false claim, and no data loss. So medium is the correct severity.

The Tier 3 portion of the finding is partly overstated (Tier 3 is explicitly "no download / metadata-only", so caption-link scanning works off ``ContentItem.text`` without a client), but the core Tier 4 gap is solid.

#### **11. user (2026-07-06T19:19:05.868Z)**

Structured output provided successfully